

# Dialogue Manager

The `DialogueManager` is the primary component that you will need to understand when working with the Dialogue System. The `DialogueManager` is responsible for managing the state of a dialogue interaction at runtime and providing Dialogue information.

The `DialogueManager` does not provide any UI or visualization of dialogue. It is intended that the frontend/UI for dialogue be created by the developer to best suit their use-case. The API of the `DialogueManager` has been designed to make this as seamless as possible.

A `DialogueManager` component can only have one active dialogue at a time. If you require multiple ongoing interactions, such as for allowing conversations to resume when leaving/returning to an NPC, then a separate instance of `DialogueManager` should be used for each.

## Beginning New Dialogue

```
void BeginDialogue(DialogueAsset dialogueAsset, string startName="")
```

To tell a `DialogueManager` to begin a new dialogue you must call `DialogueManager.BeginDialogue`. The `dialogueAsset` is the `DialogueGraph` which you wish to begin. The `startName` is an optional string field that is to be used specify the start point to use. The `startName` field is primarily used for graphs with multiple start points, as any graph with one start point is advised to leave its name blank.

This method invokes `DialogueManager.StartedDialogue` UnityEvent.

Calling this method on a `DialogueManager` that has already begun will do nothing and log an error to the console.

## Getting the Next Dialogue

```
DialogueInfo AdvanceDialogue(AdvanceContext context)  
DialogueInfo AdvanceDialogue()
```

Once a dialogue interaction has begun you must call the `DialogueManager.AdvanceDialogue` method in order to retrieve the next output for the interaction. These methods each return an instance of `DialogueInfo`, which contain all the relevant information for the next dialogue. If the `DialogueInfo` is null, this indicates that the end of the dialogue has been reached.

Calling this method prompts the `DialogueManager` to walk through the `DialogueGraph` and will perform other operations such as calling the `DSEvents` of `EventNodes`, Evaluating conditions of Redirect Options,

and setting `Variable` values for `SetVariableNodes` until it reaches a node with an output. All these operations will be performed prior to returning and in the order of the graph.

The `AdvanceContext` parameter to specify the response of the player for choice dialogue. For regular dialogue it has no effect and can be optionally excluded. You can also define your own child class of `AdvanceContext` to provide to this method. `OptionTypes` will receive an instance of this `AdvanceContext` instance when evaluating their condition, so custom `AdvanceContext` types can be used alongside custom `OptionsTypes` to express more complex behaviors.

This method invokes `DialogueManager.AdvancedDialogue` UnityEvent.

Calling this method before beginning a dialogue will return null and log an error to the console.

## Refreshing Dialogue

```
DialogueInfo RefreshDialogue(AdvanceContext context)
DialogueInfo RefreshDialogue()
```

Refreshing the dialogue can be useful if conditions or variables change over time and you would like your frontend to reflect that. Refreshing dialogue does not restart or walk backwards through the graph, it only re-returns the output of the current dialogue after re-applying `Variables` and re-evaluating available responses.

The `AdvanceContext` instance from the previous `AdvanceDialogue` or previous `RefreshDialogue` call can be reused if a new context is not provided.

Calling this method before beginning a dialogue will return null and log an error to the console.

Calling this method the first `AdvanceDialogue` has been called will return null and log an error to the console.

If either error is encountered the stored `AdvanceContext` will not be updated

## Ending a Dialogue

```
void EndDialogue()
```

This method is used if you wish to end the current dialogue prematurely. If the dialogue ends naturally this method is called before `AdvanceDialogue` returns and doesn't need to be called manually. Remember that `AdvanceDialogue` returning null means that the dialogue has ended.

This method invokes `DialogueManager.EndedDialogue` UnityEvent.

Calling this method before beginning a dialogue will return null and log an error to the console.

## Variables

The `DialogueManager` stores `Variables` using a very similar API to Unity's Animator class. You can store variables of types Bool, Int, Float, and String. Each `Variable` has a string identifier. `Variable` values are local to each `DialogueManager` and are not shared between instances.

To Assign variables you use methods in the for of `DialogueManager.SetXXX`. Setting a variable will assign the variable of that name to a new value, even if it is of a different type than the old value. `Variables` can also be set within the `DialogueManager`'s Inspector to assign initial values.

```
DialogueManager.SetString(string name, string value)
DialogueManager.SetBool(string name, bool value)
DialogueManager.SetFloat(string name, float value)
DialogueManager.SetInt(string name, int value)
```

Retrieving a value uses a similar set of methods in the form `DialogueManager.GetXXX`. Setting a `Variable` will return the currently assigned value. Attempting to retrieve a value that isn't assigned or is of the wrong type will return the default value for the respective type and log a warning to the console. If the currently running dialogue has a default value assigned and of the correct type then that value will be returned instead

```
DialogueManager.GetString(string name)
DialogueManager.GetBool(string name)
DialogueManager.GetFloat(string name)
DialogueManager.GetInt(string name)
```

Several methods also exist for setting variables using the `Variable` type. The `Variable` type is used internally to store variables and provides more flexibility in retrieving variables of unknown type.

```
bool TryGetVariable(string name, out Variable variable)
void SetVariable(string name, Variable variable)
```

## Dialogue Parameters

While the `DialogueInfo` stores all information and parameters of individual dialogue, `DialogueParameters` store information for the entire interaction. The current parameters are based on the start point that was

used to begin the interaction. The `DialogueManager` has multiple methods pertaining to these parameters.

```
Type GetDialogueParamsType()  
T GetDialogueParams<T>() where T : DialogueParameters
```

`DialogueManager.GetDialogueParamsType` returns the runtime type of the current parameters. This type will always be derived from `DialogueParameters`. The Generic Getter method can then be used to get the parameter class as a type or as the base class. If the parameters are null or of the wrong type null will be returned.

# Class DialogueManager

Namespace: [WolverineSoft.DialogueSystem](#)

Primary Component for operating the Backend of the Dialogue System. Primary functions are [Begin Dialogue\(DialogueAsset, string\)](#) and [AdvanceDialogue\(AdvanceContext\)](#). [EndDialogue\(\)](#) Is only to be used when ending an interaction prematurely. Also provides interfaces for values and keywords

```
public class DialogueManager : MonoBehaviour, IVariableContext
```

## Inheritance

[object](#)  ← DialogueManager

## Implements

[IVariableContext](#)

## Fields

### AdvancedDialogue

```
public readonly UnityEvent<AdvanceContext> AdvancedDialogue
```

#### Field Value

UnityEvent<[AdvanceContext](#)>

### EndedDialogue

```
public readonly UnityEvent EndedDialogue
```

#### Field Value

UnityEvent

# StartedDialogue

```
public readonly UnityEvent StartedDialogue
```

Field Value

UnityEvent

## optionIndexes

```
public List<int> optionIndexes
```

Field Value

[List](#) <[int](#)>

## Properties

### IsReadOnly

```
public bool IsReadOnly { get; }
```

Property Value

[bool](#)

## Methods

### AdvanceDialogue()

Retrieve the next dialogue with strict types using default context. Primarily used to get the first dialogue. Returns null if end of dialogue is reached.

```
public DialogueInfo AdvanceDialogue()
```

Returns

[DialogueInfo](#)

## AdvanceDialogue(AdvanceContext)

Retrieve the next dialogue using context about the user's interaction. Returns null if end of dialogue is reached.

```
public DialogueInfo AdvanceDialogue(AdvanceContext context)
```

Parameters

context [AdvanceContext](#)

Returns

[DialogueInfo](#)

## BeginDialogue(DialogueAsset, string)

Begin a dialogue using the DialogueAsset to be started

```
public void BeginDialogue(DialogueAsset dialogueAsset, string startName = "")
```

Parameters

dialogueAsset [DialogueAsset](#)

startName [string](#) 

## EndDialogue()

Prematurely end dialogue

```
public void EndDialogue()
```

## GetBool(string)

```
public bool GetBool(string name)
```

### Parameters

name [string](#)

### Returns

[bool](#)

## GetDialogueParamsType()

Gets type of current Dialogue Settings

```
public Type GetDialogueParamsType()
```

### Returns

[Type](#)

## GetDialogueParams<T>()

Gets the Dialogue Settings for the current dialogue

```
public T GetDialogueParams<T>() where T : DialogueParameters
```

### Returns

T

### Type Parameters

T



## GetFloat(string)

```
public float GetFloat(string name)
```

### Parameters

name [string](#)

### Returns

[float](#)

## GetInt(string)

```
public int GetInt(string name)
```

### Parameters

name [string](#)

### Returns

[int](#)

## GetString(string)

```
public string GetString(string name)
```

### Parameters

name [string](#)

### Returns

[string](#)

## RefreshDialogue()

Retrieve the information about the current dialogue again. Used if you want to account for changes in conditional choice options

```
public DialogueInfo RefreshDialogue()
```

Returns

[DialogueInfo](#)

## RefreshDialogue(AdvanceContext)

Retrieve the information about the current dialogue again. Used if you want to account for changes in conditional choice options

```
public DialogueInfo RefreshDialogue(AdvanceContext context)
```

Parameters

context [AdvanceContext](#)

Returns

[DialogueInfo](#)

## SetBool(string, bool)

```
public void SetBool(string name, bool value)
```

Parameters

name [string](#)

value [bool](#)

## SetFloat(string, float)

```
public void SetFloat(string name, float value)
```

### Parameters

name [string](#)

value [float](#)

## SetInt(string, int)

```
public void SetInt(string name, int value)
```

### Parameters

name [string](#)

value [int](#)

## SetString(string, string)

```
public void SetString(string name, string value)
```

### Parameters

name [string](#)

value [string](#)

## SetVariable(string, Variable)

```
public void SetVariable(string name, Variable variable)
```

### Parameters

name [string](#)

variable [Variable](#)

## TryGetVariable(string, out Variable)

```
public bool TryGetVariable(string name, out Variable variable)
```

### Parameters

name [string](#)

variable [Variable](#)

### Returns

[bool](#)